

1

SILLY NAME GENERATOR



The USA Network television channel once ran a detective dramedy called *Psych*, in which hyper-observant amateur sleuth Shawn Spencer solved cases while pretending to use psychic abilities. A trademark of the show was the way he would introduce his sidekick, Gus, with goofy names made up on the fly, like Galileo Humpkins, Lavender Gooms, and Bad News Marvin Barnes. As Shawn might've said, some of these names are so seriously silly they could give you a brain aneurysm, and not the good kind!

Project #1: Generating Pseudonyms

In this warm-up project, you'll write a simple Python program that generates nutty names by randomly combining first names and surnames. With any luck, you'll produce a plethora of aliases that would make any sidekick proud. You'll also review best-practice coding guidelines and apply external programs that will help you write code that conforms to those guidelines.

Psych not your thing? Replace the names in my list in the code with your own jokes or theme. You could just as easily turn this project into a *Game of Thrones* name generator, or perhaps you want to discover your very own "Benedict Cumberbatch" name; my favorite is Bendylick Cricketbat.

THE OBJECTIVE

Randomly generate funny sidekick names using Python code that conforms to established style guidelines.

Planning and Designing a Project

Planning time is never wasted time. It doesn't matter whether you're programming for fun or profit; at some point, you'll need to estimate—fairly accurately—how long the project will take, what obstacles you may encounter, and what tools and resources you'll need to do the work. And to accomplish that, you'll need to know what you're trying to create in the first place!

A successful manager once told me that his secret was simply to ask lots of questions: *What* are you trying to do? *Why* are you doing it? Why are you doing it *this way*? How much *time* do you have? How much *money*? Answering these questions is extremely helpful to the design process and gives you a clear line of sight.

In his book *Think Python, 2nd Edition* (O'Reilly, 2015), Allen Downey describes two types of software development plans: “prototype and patch” and “designed development.” With prototype and patch, you start with a simple program and then use *patches*, or edited code, to handle problems encountered in testing. This can be a good approach when you're working through a complex problem you don't understand very well. But it can also produce complicated and unreliable code. If you have a clear view of the problem and how you want to solve it, you should use a designed development plan to avoid future issues and their subsequent patches. This approach can make coding easier and more efficient, and it typically leads to stronger and more reliable code.

For all the projects in this book, you'll start with a clearly defined problem or goal that will form the basis of your design decisions. Then we'll discuss strategy to better understand the issues and create a designed development plan.

The Strategy

You'll start with two lists—first and last—of funny names. The lists will be relatively short, so they won't be memory intensive, won't need to be dynamically updated, and shouldn't present any runtime issues. Since all you'll need to do is read names from the list, you'll use a tuple as a container.

With your two tuples of names, you'll generate new names—pairing a first name with a last—at the touch of a button. That way, the user can easily repeat the process until a sufficiently funny name appears.

You should also highlight the name in the interpreter window somehow so it stands out from the command prompts. The IDLE shell doesn't provide many font options, but you probably know—all too well—that errors appear in red. The default for the `print()` function is the standard output, but with the `sys` module loaded, you can redirect the output to the error channel, with its trademark red coloring, using the `file` parameter:

```
print(something, file=sys.stderr).
```

Finally, you'll determine what style recommendations exist for Python programming. These guidelines should address not only the code but also documentation embedded within the code.

The Pseudocode

“You can always count on the Americans to do the right thing after they have tried everything else.” That quote, weakly linked to Winston Churchill, sums up the way many people approach writing pseudocode.

Pseudocode is a high-level, informal way to describe computer programs using structured English or any human language. It should resemble a simplified programming language and include keywords and proper indentations. Developers use it to ignore all the arcane syntax of true programming languages and focus on the underlying logic. Despite its widespread use, pseudocode has no official standards—only guidelines.

If you find you've hacked your way into frustration, it may be because you didn't take the time to write pseudocode. I am a true believer in it, as pseudocode has—without fail—guided me to solutions when I was otherwise lost in the woods. Consequently, you'll use some form of pseudocode

in most of the projects in this book. At the very least, I hope you'll see its utility, but I also hope you'll develop the discipline to write it in your own projects.

A very high-level pseudocode for our funny name generator could look like this:

```
Load a list of first names
Load a list of surnames
Choose a first name at random
Assign the name to a variable
Choose a surname at random
Assign the name to a variable
Print the names to the screen in order and in red font
Ask the user to quit or play again
If user plays again:
    repeat
If user quits:
    end and exit
```

Unless you're trying to pass a programming class or provide clear instructions to others, focus on the *purpose* of the pseudocode; don't worry about slavishly complying with the (nonstandard) guidelines for writing it. And don't stop with programming—you can apply the pseudocode process to much more. Once you get the hang of it, you might find it helps you complete other tasks like doing your taxes, planning your investments, building a house, or getting ready for a camping trip. It's a great way to focus your thinking and carry programming successes over into real life. If only Congress would use it!

The Code

Listing 1-1 is the code for the funny name generator, *pseudonyms.py*, which compiles and prints a list of pseudonyms from two tuples of names. If you don't want to type all the names, you can type a subset or download the code from <https://nostarch.com/impracticalpython/>.

pseudonyms.py

```

❶ import sys, random

❷ print("Welcome to the Psych 'Sidekick Name Picker.'\n")
   print("A name just like Sean would pick for Gus:\n\n")

first = ('Baby Oil', 'Bad News', 'Big Burps', "Bill 'Beenie-Weenie'",
         "Bob 'Stinkbug'", 'Bowel Noises', 'Boxelder', "Bud 'Lite' ",
         'Butterbean', 'Buttermilk', 'Buttocks', 'Chad', 'Chesterfield',
         'Chewy', 'Chigger", "Cinnabuns", 'Cleat', 'Cornbread', 'Crab Meat',
         'Crapps', 'Dark Skies', 'Dennis Clawhammer', 'Dicman', 'Elphonso',
         'Fancypants', 'Figgs', 'Foncy', 'Gootsy', 'Greasy Jim', 'Huckleberry
         'Huggy', 'Ignatious', 'Jimbo', "Joe 'Pottin Soil'", 'Johnny',
         'Lemongrass', 'Lil Debil', 'Longbranch', '"Lunch Money"',
         'Mergatroid', '"Mr Peabody"', 'Oil-Can', 'Oinks', 'Old Scratch',
         'Ovaltine', 'Pennywhistle', 'Pitchfork Ben', 'Potato Bug',
         'Pushmeet', 'Rock Candy', 'Schlomo', 'Scratchensniff', 'Scut',
         "Sid 'The Squirts'", 'Skidmark', 'Slaps', 'Snakes', 'Snoobs',
         'Snorki', 'Soupcan Sam', 'Spitzitout', 'Squids', 'Stinky',
         'Storyboard', 'Sweet Tea', 'TeeTee', 'Wheezy Joe',
         "Winston 'Jazz Hands'", 'Worms')

last = ('Appleyard', 'Bigmeat', 'Bloominshine', 'Boogerbottom',
        'Breedslovetrout', 'Butterbaugh', 'Clovenhoof', 'Clutterbuck',
        'Cocktoasten', 'Endicott', 'Fewhairs', 'Gooberdapple', 'Goodensmith',
        'Goodpasture', 'Guster', 'Henderson', 'Hooperbag', 'Hoosenater',
        'Hootkins', 'Jefferson', 'Jenkins', 'Jingley-Schmidt', 'Johnson',
        'Kingfish', 'Listenbee', "M'Bembo", 'McFadden', 'Moonshine', 'Nettles
        'Noseworthy', 'Olivetti', 'Outerbridge', 'Overpeck', 'Overturf',
        'Oxhandler', 'Pealike', 'Pennywhistle', 'Peterson', 'Pieplow',
        'Pinkerton', 'Porkins', 'Putney', 'Quakenbush', 'Rainwater',
        'Rosenthal', 'Rubbins', 'Sackrider', 'Snuggleshine', 'Splern',
        'Stevens', 'Stroganoff', 'Sugar-Gold', 'Swackhamer', 'Tippins',
        'Turnipseed', 'Vinaigrette', 'Walkingstick', 'Wallbanger', 'Weewax',
        'Weiners', 'Whipkey', 'Wigglesworth', 'Wimplesnatch', 'Winterkorn',
        'Woolysocks')

```

```

❸ while True:

```

```

❷ firstNames = random.choice(first)

❸ lastNames = random.choice(last)

    print("\n\n")
❹ print("{} {}".format(firstNames, lastNames), file=sys.stderr)
    print("\n\n")

❺ try_again = input("\n\nTry again? (Press Enter else n to quit)\n ")
    if try_again.lower() == "n":
        break

❻ input("\nPress Enter to exit.")

```

Listing 1-1: Generates silly pseudonyms from tuples of names

First, import the `sys` and `random` modules ❶. You’ll use `sys` to access the system-specific error message functionality, so you can color your output an eye-catching red in the IDLE window. And `random` lets you pick, at random, items from your name lists.

The `print` statements at ❷ introduce the program to the user. The newline command `\n` forces a new line, and single quotes `' '` allow you to use quotes in the printout without having to resort to the backslash escape character, which would reduce code readability.

Next, define your tuples of names. Then initiate the `while` loop ❸. Setting `while = True` basically means “Keep running until I tell you to stop.” Eventually, you’ll use a `break` statement to end the loop.

The loop starts by choosing a name from the `first` tuple at random and then assigns that name to the variable `firstName` ❹. It uses the `random` module’s `choice` method to return a random element from a nonempty sequence—in this case, the tuple of first names.

Next, choose a surname at random from the `last` tuple and assign it to the variable `lastName` ❺. Now that you have both names, print them and trick IDLE into using the red “error” font by supplying the optional argument

`file=sys.stderr` to the `print` statement ❹. Use the newer string format *method*, rather than the older string format *operator* (`%`), to convert the name variables to a string. To read more about the new method, see <https://docs.python.org/3.7/library/string.html>.

Once the name is displayed, ask the user to choose to play again or quit, using `input` to provide the instruction in quotes. In this case, include a few blank lines as well to make the funny name more obvious in the IDLE window. If the user responds by pressing the ENTER key, nothing is returned to the `try_again` variable ❺. With nothing returned, the condition on the `if` statement isn't met, the `while` loop continues, and a new name is printed. If the user instead presses the N key, the `if` statement results in a `break` command, and the loop ends because the `while` statement no longer evaluates to `True`. Use the lowercase string method `.lower()` to mitigate the player's CAPS LOCK key being engaged. In other words, it doesn't matter whether the user inputs a lowercase or uppercase N, because the program will always read it as lowercase.

Finally, ask the user to exit by pressing the ENTER key ❻. Pressing ENTER doesn't assign the return value of `input()` to a variable, the program ends, and the console window closes. Pressing F5 in the IDLE editor window executes the completed program.

This code works, but working isn't enough—programs in Python should work with *style*.

Using the Python Community's Style Guide

According to the *Zen of Python* (<https://www.python.org/dev/peps/pep-0020/>), “There should be one—and preferably only one—obvious way to do something.” In the spirit of providing a single obvious “right way” of doing things and building consensus around these practices, the Python community releases *Python Enhancement Proposals*, which are coding conventions for the Python code comprising the standard library in the main Python distribution. The most important of these is *PEP 8*, a style guide for Python programming. PEP 8 evolves over time as new conventions are identified and past ones are rendered obsolete by changes in the language.

PEP 8 (<https://www.python.org/dev/peps/pep-0008/>) sets standards for naming conventions; use of blank lines, tabs, and spaces; maximum line length; comments; and so on. The goal is to improve the readability of code and make it consistent across a wide spectrum of Python programs. When you start programming, you should strive to learn and follow the accepted conventions, before bad habits become engrained. The code in this book will conform closely to PEP 8, but I have overridden some conventions (by using less commented code, fewer blank lines, and shorter docstrings, for example) in deference to the publishing industry.

Standardized names and procedures are especially important when you're working in cross-functional teams. A lot can get lost in translation between scientists and engineers, as in 1999, when engineers lost the Mars Climate Orbiter because different teams used different measurement units. For almost two decades, I built computer models of the earth that were transferred to an engineering function. The engineers used scripts to load these models into their own proprietary software. They would share these scripts among projects for efficiency and to help the inexperienced. Since these "command files" were customized to each project, the engineers were understandably annoyed when attribute names changed during model updates. In fact, one of their internal guidelines was "Beg, bribe, or bully your modeler into using consistent property names!"

Checking Your Code with Pylint

You should become familiar with PEP 8, but you'll still make mistakes, and comparing your code to the guide is a major drag. Luckily, programs such as Pylint, pycodestyle, and Flake8 can help you easily follow the PEP 8 style recommendations. For this project, you'll use Pylint.

Installing Pylint

Pylint is a source code, bug, and quality checker for the Python programming language. To download a free copy, go to <https://www.pylint.org/#install> and find the install button for your platform. This button will show the command for installing Pylint. For example, in Windows, go to the folder that contains your copy of Python (such as `C:\Python35`), use SHIFT-right-click to open a context menu, and then

click either **open command window here** or **open PowerShell window here**, depending on which version of Windows you're using. Run `pip install pylint` (pip3 if Python 2 *and* 3 are installed).

Running Pylint

In Windows, Pylint is run from a command window or, for newer systems, the PowerShell (you open both by using SHIFT-right-click in the folder containing the Python module you want to check). Type `pylint filename` to run the program (see **Figure 1-1**). The `.py` extension is optional, and your directory path will vary from the one shown. On macOS or another Unix-based system, use the terminal emulator.

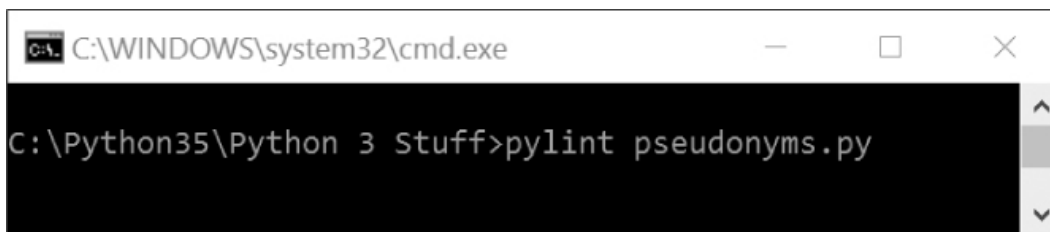


Figure 1-1: The Windows command window with the command to run Pylint

The command window will display the Pylint results. Here's a sample of a useful output:

```
C:\Python35\Python 3 Stuff\Psych>pylint pseudonyms.py
No config file found, using default configuration
***** Module pseudonyms
C: 45, 0: No space allowed around keyword argument assignment
      print(firstName, lastName, file = sys.stderr)
                                   ^ (bad-whitespace)
C:  1, 0: Missing module docstring (missing-docstring)
C:  2, 0: Multiple imports on one line (sys, random) (multiple-imports)
C:  7, 0: Invalid constant name "first" (invalid-name)
C: 23, 0: Invalid constant name "last" (invalid-name)
C: 40, 4: Invalid constant name "firstName" (invalid-name)
C: 42, 4: Invalid constant name "lastName" (invalid-name)
C: 48, 4: Invalid constant name "try_again" (invalid-name)
```

The capital letter at the start of each line is a message code. For example, `C: 15, 0` refers to a coding standard violation in line 15, column 0. You can reference the following key for the various Pylint message codes:

R Refactor for a “good practice” metric violation

C Convention for coding standard violation

W Warning for stylistic problems or minor programming issues

E Error for important programming issues (i.e., most probably a bug)

F Fatal for errors that prevent further processing

Pylint will end its report by grading your program’s conformance to PEP 8. In this case, your code received a 4 out of 10:

```
Global evaluation
-----
Your code has been rated at 4.00/10 (previous run: 4.00/10, +0.00)
```

Handling False Constant Name Errors

You might have noticed that Pylint incorrectly assumes all variable names in the global space refer to constants, and should therefore be in all caps. You can work around this shortcoming in a number of ways. The first is to embed your code in a `main()` function (as shown in **Listing 1-2**); that way, it’s out of the global space.

```
def main():
    some indented code
    some indented code
    some indented code
❶ if __name__ == "__main__":
    ❷ main()
```

Listing 1-2: Defines and calls a `main()` function

The `__name__` variable is a special built-in variable that you can use to evaluate whether a program is being run in stand-alone mode or as an imported module; remember that a module is just a Python program used inside of another Python program. If you run the program directly, `__name__` is set to `__main__`. In **Listing 1-2** `__name__` is used to ensure that, when the program is imported, the `main()` function isn't run until you intentionally call it, but when you run the program directly, the condition in the `if` statement is met ❶ and `main()` is automatically called ❷. You don't always need this convention. For example, if your code just defines a function, you can load it as a module and call it without the need for `__name__`.

Let's embed everything in *pseudonyms.py*, except for the `import` statement, under a `main()` function and then embed the `main()` function call under an `if` statement, as in **Listing 1-2**. You can make the changes yourself or download the *pseudonyms_main.py* program from the website. Rerun Pylint. You should get the following results in your command window.

```
C:\Python35\Python 3 Stuff\Psych>pylint pseudonyms_main
No config file found, using default configuration
***** Module pseudonyms_main
C: 47, 0: No space allowed around keyword argument assignment
        print(firstName, lastName, file = sys.stderr)
                                   ^ (bad-whitespace)
C:  1, 0: Missing module docstring (missing-docstring)
C:  2, 0: Multiple imports on one line (sys, random) (multiple-imports)
C:  4, 0: Missing function docstring (missing-docstring)
C: 42, 8: Invalid variable name "firstName" (invalid-name)
C: 44, 8: Invalid variable name "lastName" (invalid-name)
```

Now those annoying comments about invalid constant names have disappeared, but you aren't out of the woods yet. As much as I like them, Python conventions don't allow for *camel case* names, like `firstName`.

Configuring Pylint

When evaluating small scripts, I prefer to use the Pylint defaults and ignore the false "constant name" errors. I also like to run the option `-rn`

(short for `-reports=n`) to suppress the large volume of extraneous statistics that Pylint returns:

```
C:\Python35\Python 3 Stuff\Psych>pylint -rn pseudonyms_main.py
```

Note that using `-rn` will disable the code-grading option.

Another issue with Pylint is that its maximum line length default is 100 characters but PEP 8 recommends 79 characters. To comply with PEP 8, you can run Pylint with the following option:

```
C:\Python35\Python 3 Stuff\Psych>pylint --max-line-length=79 pseudonyms_main
```

Now you'll see that indenting the names for the `main()` function caused some lines to exceed the guidelines:

```
C: 12, 0: Line too long (80/79) (line-too-long)
C: 14, 0: Line too long (83/79) (line-too-long)
--snip--
```

You probably don't want to configure Pylint every time you run it, and fortunately, you don't have to. Instead, you can make your own customized configuration file using the command `--generate-rcfile`. For example, to suppress reporting and set the maximum line length to 79, enter the following into your command prompt:

```
your pathname>pylint -rn --max-line-length=79 --generate-rcfile > name.pylintrc
```

Put the changes you want before the `--generate-rcfile > name.pylintrc` statement and provide your own name before the `.pylintrc` extension. You can create a configuration file either stand-alone, as just shown, or at the same time you evaluate a Python program. The `.pylintrc` file is automatically saved in your current working directory, though there is an option for adding a directory path (see <https://pylint.org> and

https://pylint.readthedocs.io/en/latest/user_guide/run.html for more details).

To reuse your custom configuration file, use the `--rcfile` option followed by the name of your personal configuration file and the name of the program you're evaluating. For example, to run *myconfig.pylintrc* on the *pseudonyms_main.py* program, enter the following:

```
C:\Python35\Python 3 Stuff\Psych>pylint --rcfile myconfig.pylintrc pseudonyms_main
```

Describing Your Code with Docstrings

Pylint identifies that the *pseudonyms_main.py* program is missing a docstring. According to the PEP 257 style guide (<https://www.python.org/dev/peps/pep-0257/>), a *docstring* is a string literal that occurs as the first statement in a module, function, class, or method definition. A docstring is basically a short description of what your code does, and it may include specifics on aspects of the code such as required inputs. Here, in triple quotes, is an example of a single-line docstring for a function:

```
def circ(r):  
    """Return the circumference of a circle with radius of r."""  
    c = 2 * r * math.pi  
    return c
```

The preceding docstring simply states what the function does, but docstrings can be longer and include more information. For instance, the following is a multiline docstring for the same function that displays information about the function's input and output:

```
def circ(r):  
    """Return the circumference of a circle with radius of r.  
  
    Arguments:  
    r - radius of circle
```

```
Returns:
    float: circumference of circle
"""
c = 2 * r * math.pi
return c
```

Unfortunately, docstrings are person-, project-, and company-specific things, and you can find a lot of conflicting guidance. Google has its own format and an excellent style guide. Some members of the scientific community use a NumPy docstring standard. And *reStructuredText* is a popular format used mainly in conjunction with Sphinx—a tool that uses docstrings to generate documentation for Python projects in formats such as HTML and PDF. If you’ve ever read the docs (<https://readthedocs.org/>) for a Python module, then you’ve seen Sphinx in action. You can find links to guides for some of these different styles in “**Further Reading**” on [page 14](#).

You can check how well your docstrings conform to the PEP 257 conventions with a free tool called pydocstyle. To install it in Windows or any other operating system, open a command window and run `pip install pydocstyle` (use `pip3` if both Python 2 and 3 are installed).

To run pydocstyle, open a command window in the folder containing the code you want to check. If you don’t specify a filename, pydocstyle will run on *all* the Python programs in the folder and give you feedback:

```
C:\Python35\Python 3 Stuff\Psych>pydocstyle
.\OLD_pseudonyms_main.py:1 at module level:
    D100: Missing docstring in public module
.\OLD_pseudonyms_main.py:4 in public function `main`:
    D103: Missing docstring in public function
.\ pseudonyms.py:1 at module level:
    D100: Missing docstring in public module
.\ pseudonyms_main_broken.py:1 at module level:
    D200: One-line docstring should fit on one line with quotes (found 2)
.\ pseudonyms_main_broken.py:6 in public function `main`:
```

```
D205: 1 blank line required between summary line and description
(found 0)
```

If you specify a file with no docstring issues, pydocstyle will return nothing:

```
C:\Python35\Python 3 Stuff\Psych>pydocstyle pseudonyms_main_fixed.py

C:\Python35\Python 3 Stuff\Psych>
```

I'll use fairly simple docstrings in all of the projects in this book in order to reduce visual noise in the annotated code. Feel free to expand on these if you wish to practice. You can always check your results with pydocstyle.

Checking Your Code Style

When I was growing up, my uncle would drive from our rural town to a larger city to have his hair “styled.” I never understood how that was different from a regular haircut, but I do know how to “style” our funny name generator code so that it complies with PEP 8 and PEP 257.

Make a copy of *pseudonyms_main.py* called *pseudonyms_main_fixed.py* and immediately evaluate it with Pylint using this command:

```
your_path>pylint --max-line-length=79 pseudonyms_main_fixed
```

Don't suppress the report using `-rn`. You should see this output at the bottom of the command window:

```
Global evaluation
-----
Your code has been rated at 3.33/10
```

Now correct the code based on the Pylint output. In the following example, I have provided the corrections in **bold**. I made changes to the name

tuples to correct for line-length issues. You can also download the corrected code, *pseudonyms_main_fixed.py*, from the book's resources at <https://www.nostarch.com/impracticalpython/>.

pseudonyms_main_fixed.py

```
"""Generate funny names by randomly combining names from 2 separate lists."""
import sys
import random

def main():
    """Choose names at random from 2 tuples of names and print to screen."""
    print("Welcome to the Psych 'Sidekick Name Picker.'\n")
    print("A name just like Sean would pick for Gus:\n\n")

    first = ('Baby Oil', 'Bad News', 'Big Burps', "Bill 'Beenie-Weenie'",
             "Bob 'Stinkbug'", 'Bowel Noises', 'Boxelder', "Bud 'Lite'",
             'Butterbean', 'Buttermilk', 'Buttocks', 'Chad', 'Chesterfield',
             'Chewy', 'Chigger', 'Cinnabuns', 'Cleat', 'Cornbread',
             'Crab Meat', 'Crapps', 'Dark Skies', 'Dennis Clawhammer',
             'Dicman', 'Elphonso', 'Fancypants', 'Figgs', 'Foncy', 'Gootsy',
             'Greasy Jim', 'Huckleberry', 'Huggy', 'Ignatious', 'Jimbo',
             "Joe 'Pottin Soil'", 'Johnny', 'Lemongrass', 'Lil Debil',
             'Longbranch', '"Lunch Money"', 'Mergatroid', '"Mr Peabody"',
             'Oil-Can', 'Oinks', 'Old Scratch', 'Ovaltine', 'Pennywhistle',
             'Pitchfork Ben', 'Potato Bug', 'Pushmeet', 'Rock Candy',
             'Schlomo', 'Scratchensniff', 'Scut', "Sid 'The Squirts'",
             'Skidmark', 'Slaps', 'Snakes', 'Snoobs', 'Snorki', 'Soupcan Sam',
             'Spitzitout', 'Squids', 'Stinky', 'Storyboard', 'Sweet Tea',
             'TeeTee', 'Wheezy Joe', "Winston 'Jazz Hands'", 'Worms')

    last = ('Appleyard', 'Bigmeat', 'Bloominshine', 'Boogerbottom',
            'Breedslovetrou', 'Butterbaugh', 'Clovenhoof', 'Clutterbuck',
            'Cocktoasten', 'Endicott', 'Fewhairs', 'Gooberdapple',
            'Goodensmith', 'Goodpasture', 'Guster', 'Henderson', 'Hooperbag',
            'Hoosenater', 'Hootkins', 'Jefferson', 'Jenkins',
            'Jingley-Schmidt', 'Johnson', 'Kingfish', 'Listenbee', "M'Bembo",
            'McFadden', 'Moonshine', 'Nettles', 'Noseworthy', 'Olivetti',
```



```

        'Outerbridge', 'Overpeck', 'Overturf', 'Oxhandler', 'Pealike',
        'Pennywhistle', 'Peterson', 'Pieplow', 'Pinkerton', 'Porkins',
        'Putney', 'Quakenbush', 'Rainwater', 'Rosenthal', 'Rubbins',
        'Sackrider', 'Snuggleshine', 'Splern', 'Stevens', 'Stroganoff',
        'Sugar-Gold', 'Swackhamer', 'Tippins', 'Turnipseed',
        'Vinaigrette', 'Walkingstick', 'Wallbanger', 'Weewax', 'Weiners',
        'Whipkey', 'Wigglesworth', 'Wimplesnatch', 'Winterkorn',
        'Woolysocks')

while True:
    first_name = random.choice(first)
    last_name = random.choice(last)

    print("\n\n")
    # Trick IDLE by using "fatal error" setting to print name in red.
    print("{} {}".format(first_name, last_name), file=sys.stderr)
    print("\n\n")

    try_again = input("\n\nTry again? (Press Enter else n to quit)\n ")

    if try_again.lower() == "n":
        break

    input("\nPress Enter to exit.")

if __name__ == "__main__":
    main()

```

Pylint gives the revised code a grade of 10 out of 10:

```
Global evaluation
```

```
-----
```

```
Your code has been rated at 10.00/10 (previous run: 3.33/10, +6.67)
```

As you saw in the previous section, running `pydocstyle` on `pseudonyms_main_fixed.py` yields no errors, but don't be fooled into think-

ing that means it's good or even adequate. For example, this docstring also passes: `"""ksjkdls lskjds kjs jdi wllk sijkljs dsdw noiu sss."""`

It's hard to write sparse, succinct, and truly useful docstrings and comments. PEP 257 will help with docstrings, but comments are more freestyle and “open range.” Too many comments create visual noise, can be off-putting to the user, and shouldn't be needed, as well-written code is largely self-documenting. Good reasons for adding comments include clarifying intent and heading off potential user errors, such as when specific measurement units or input formats are required. To find the right balance in commenting, take note of good examples when you run across them. Also, think about what you would want to see if you had to pick up your own code after a five-year hiatus!

Pylint and pydocstyle are easy to install, are easy to run, and will help you learn and comply with the accepted coding standards of the Python community. Running your code through Pylint prior to posting it on web forums is also a good practice when you're seeking help and should prompt “kinder, gentler” responses!

Summary

You should now know how to write code and documentation that conforms to the Python community's expectations. More importantly, you've generated some seriously funny names for a sidekick, gangster, informant, whoever. Here are a few of my favorites:

Pitchfork Ben Pennywhistle	'Bad News' Bloominshine
----------------------------	-------------------------

Chewy Stroganoff	'Sweet Tea' Tippins
------------------	---------------------

Spitzitout Winterkorn	Wheezy Joe Jenkins
-----------------------	--------------------

'Big Burps' Rosenthal	Soupcan Sam Putney
-----------------------	--------------------

Bill 'Beenie-Weenie' Clutterbuck	Greasy Jim Wigglesworth
----------------------------------	-------------------------

Dark Skies Jingley-Schmidt	Chesterfield Walkingstick
----------------------------	---------------------------

Potato Bug Quakenbush

Jimbo Woollysocks

Worms Endicott

Fancypants Pinkerton

Cleet Weiners

Dicman Overpeck

Ignatious Outerbridge

Buttocks Rubbins

Further Reading

For a clickable version of these resources, visit

<https://www.nostarch.com/impracticalpython/>.

Pseudocode

Descriptions of some fairly formal pseudocode standards can be found at

http://users.csc.calpoly.edu/~jdalbey/SWE/pdl_std.html and

<http://www.slideshare.net/sabiksabz/pseudo-code-basics/>.

Style Guides

Here's a list of style guides you can reference when creating Python programs.

- The PEP 8 style guide can be found at <https://www.python.org/dev/peps/pep-0008/>.
- The PEP 257 guidelines can be found at <https://www.python.org/dev/peps/pep-0257/>.
- Google has its own format and style guide at <https://google.github.io/styleguide/pyguide.html>.
- Examples of Google style can be found at https://sphinxcontrib-napoleon.readthedocs.io/en/latest/example_google.html.
- NumPy docstring standards are at <https://numpydoc.readthedocs.io/en/latest/>.
- NumPy docstrings examples can be found at https://sphinxcontrib-napoleon.readthedocs.io/en/latest/example_numpy.html.
- You can find out about reStructuredText at <https://docs.python.org/devguide/documenting.html>,

<https://docs.python.org/3.1/documenting/rest.html>, and
<https://wiki.python.org/moin/reStructuredText/>.

- *The Hitchhiker's Guide to Python* (<http://docs.python-guide.org/en/latest/writing/style/>) contains a section on code styles and *autopep8*, which will automatically reformat code for PEP 8 (to a point).
- *Effective Python* by Brett Slatkin (Addison-Wesley, 2015) contains a useful section on documenting programs.

Third-Party Modules

The following are some resources for using third-party modules.

- Details on Pylint are at <https://docs.pylint.org/en/1.8/tutorial.html>.
- Details on pydocstyle can be found at <http://www.pydocstyle.org/en/latest/>.

Practice Projects

Try out these projects for working with strings. My own solutions are available in the appendix.

Pig Latin

To form Pig Latin, you take an English word that begins with a consonant, move that consonant to the end, and then add “ay” to the end of the word. If the word begins with a vowel, you simply add “way” to the end of the word. One of the most famous Pig Latin phrases of all time is “ixnay on the ottenray,” uttered by Marty Feldman in Mel Brooks’s comedic masterpiece *Young Frankenstein*.

Write a program that takes a word as input and uses indexing and slicing to return its Pig Latin equivalent. Run Pylint and pydocstyle on your code and correct any style mistakes. You can find a solution in the appendix or download *pig_latin_practice.py* from

<https://www.nostarch.com/impracticalpython/>.

Poor Man's Bar Chart

The six most commonly used letters in the English language can be remembered with the mnemonic “etaoin” (pronounced *eh-tay-oh-in*). Write a Python script that takes a sentence (string) as input and returns a simple bar chart–type display as in **Figure 1-2**. Hint: I used a dictionary data structure and two modules that I haven’t covered yet, `pprint` and `collections/defaultdict`.



```
Python 3.5.2 Shell
File Edit Shell Debug Options Window Help
You may need to stretch console window if text wrapping occurs.

text = Like the castle in its corner in a medieval game, I foresee terrible trouble and I stay here just
the same.

defaultdict(<class 'list'>,
            {'a': ['a', 'a', 'a', 'a', 'a', 'a', 'a'],
             'b': ['b', 'b'],
             'c': ['c', 'c'],
             'd': ['d', 'd'],
             'e': ['e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e'],
             'f': ['f'],
             'g': ['g'],
             'h': ['h', 'h', 'h'],
             'i': ['i', 'i', 'i', 'i', 'i', 'i', 'i', 'i'],
             'j': ['j'],
             'k': ['k'],
             'l': ['l', 'l', 'l', 'l', 'l'],
             'm': ['m', 'm', 'm'],
             'n': ['n', 'n', 'n', 'n'],
             'o': ['o', 'o', 'o'],
             'r': ['r', 'r', 'r', 'r', 'r', 'r', 'r'],
             's': ['s', 's', 's', 's', 's', 's'],
             't': ['t', 't', 't', 't', 't', 't', 't', 't'],
             'u': ['u', 'u'],
             'v': ['v'],
             'y': ['y']})

>>>
```

Figure 1-2: Bar chart–like output of the `ETAOIN_practice.py` program in the appendix

Challenge Projects

No solutions are provided for challenge projects. You’re on your own with these!

Poor Foreign Man’s Bar Chart

Use an online translator to change your text into another Latin-based writing system (such as Spanish or French), rerun your code from the Poor Man’s Bar Chart, and compare the results. For example, a Spanish version of the text in **Figure 1-2** yields the results in **Figure 1-3**.



```
Python 3.5.2 Shell
File Edit Shell Debug Options Window Help
You may need to stretch console window if text wrapping occurs.

text = Al igual que el castillo en la esquina en un juego medieval, preveo terribles problemas y me quedo
aqui lo mismo.

defaultdict(<class 'list'>,
  {'a': ['a', 'a', 'a', 'a', 'a', 'a', 'a', 'a'],
   'b': ['b', 'b'],
   'c': ['c'],
   'd': ['d', 'd'],
   'e': ['e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e', 'e'],
   'q': ['q', 'q'],
   'i': ['i', 'i', 'i', 'i', 'i', 'i'],
   'j': ['j'],
   'l': ['l', 'l', 'l', 'l', 'l', 'l', 'l', 'l', 'l', 'l'],
   'm': ['m', 'm', 'm', 'm', 'm'],
   'n': ['n', 'n', 'n', 'n'],
   'o': ['o', 'o', 'o', 'o', 'o', 'o', 'o'],
   'p': ['p', 'p'],
   'r': ['r', 'r', 'r', 'r'],
   's': ['s', 's', 's', 's', 's'],
   't': ['t', 't'],
   'u': ['u', 'u', 'u', 'u', 'u', 'u'],
   'v': ['v', 'v'],
   'y': ['y']})

>>> |
```

Figure 1-3: The results of running EATOIN_challenge.py on a Spanish translation of the text in **Figure 1-2**

Twice as many *L*s and three times as many *U*s appear in the Spanish sentence. To make the bar charts for different inputs directly comparable, change the code so every letter of the alphabet has a key and is displayed even if there are no values.

The Middle

Rewrite the funny name generator code to include middle names. First, create a new `middle_name` tuple, then split apart existing first name–middle name pairs (such as “Joe ‘Pottin Soil’” or “Sid ‘The Squirts’”) and add them to the tuple. You should also move some obvious nicknames (like “Oil Can”) to your `middle_name` tuple. Finally, add some new middle names (such as “The Big News,” or “Grunts,” or “Tinkie Winkie”). Use Python’s `random` module so that a middle name is chosen only one-half or one-third of the time.

Something Completely Different

Start your own list of funny names and add to the funny name generator. Hint: movie credits are a rich hunting ground!